

Моите първи C++ програми

Подготовка по информатика за 5. клас
Първи два учебни часа и самоподготовка

Ахмед Матем Ахмед · 2026/2027

Как да използваш презентацията

Слайдове 3–26: първи час. Инсталирането на слайдове 7–10 е справка и за вкъщи.

Слайдове 27–40: втори час, задачи 02–04 и проверка на програмите.

Слайдове 41–46: домашно, допълнителна задача и подсказки.

Слайдове 47–52: решения. Отвори ги след собствен опит.

Слайдове 53–68: продължение за следващото занятие. Слайд 69: проверка на напредъка.

Нашата цел за днес

Създавам проект и изпълнявам C++ програма.

Разпознавам входа, изчислението и изхода.

Използвам цели числа, `cin`, `cout`, `+`, `-` и `*`.

Проверявам с различни данни и запазвам своя `.cpp` файл.

Не е нужно веднага да запомниш целия начален шаблон.

Какво е състезателно програмиране

Получаваш задача с точно описани входни данни и търсен резултат.

Измисляш начин за решаване и го записваш като програма.

Проверяващата система изпълнява програмата с различни тестове.

Един верен пример е начало. Решението трябва да работи за всички допустими данни.

Днешните задачи са учебни упражнения, а не официална състезателна тема.

Алгоритъм: точни съпки

Задача: в две кутии има 7 и 5 молива. Колко са общо?

1. Научи броя на моливите във всяка кутия.
2. Събери двата броя: $7 + 5 = 12$.
3. Съобщи резултата: 12.

Същите съпки работят и за 2 и 9 молива. Променят се данните.

Вход: 7 5 Изчисление: $7 + 5$ Изход: 12

Език, среда и компилатор

C++ е езикът, на който ще записваме програмите.

Code::Blocks е средата, в която пишем и стартираме кода.

Компилаторът превежда кода в изпълнима програма и съобщава за синтактични грешки.

При Windows ще използваме пакет с MinGW/GCC, който включва и компилатор.

Инсталиране 1: подходящият файл

Инструкциите са за Windows. При друга система потърси помощ от учителя.

Отвори официалната страница: codeblocks.org/downloads/binaries/

В раздел Windows избери `codeblocks-25.03mingw-setup.exe`.

Думата `mingw` в името е важна: този пакет съдържа компилатор.

Използвай връзката за изтегляне в реда на файла.

Версията е проверена на 23.09.2026. При бъдеща версия отново търси пакет с MinGW.

Инсталиране 2: съветникът

1. Отвори изтегляния .exe файл. При искане за администраторски права потърси възрастен.
2. Продължи с Next и прочети лицензионните условия.
3. В избора на компоненти остави включени средствата MinGW/GCC.
4. Запази предложената папка и избери Install.
5. Изчакай края и стартирай Code::Blocks. Надписите може да се различават по версия.

Инсталиране 3: първо стартиране

Ако се появи списък с намерени компилатори, избери GNU GCC Compiler.

Потвърди избора. Следващата проверка е истинска компилация.

Създай папка C:\Olympiad\Week01 или Olympiad\Week01 в Documents.

За всяка задача прави отделен проект: task01, task02 и т.н.

Кратките имена на латиница улесняват намирането на файловете.

Когато компилаторът липсва

Симптом: „compiler not found“ или „compiler setup is invalid“.

Отвори Settings > Compiler и избери GNU GCC Compiler.

В Toolchain executables натисни Auto-detect.

Ако не го намери, провери дали си инсталирал пакета с mingw. Потърси помощ от учителя.

След поправката компилирай отново. Самото отваряне на редактора не доказва, че всичко работи.

Нов проект: стъпки 1–4

1. File > New > Project.
2. Избери Console application и натисни Go.
3. На началния екран натисни Next, ако се появи.
4. Избери C++ и продължи с Next.

Console application е програма с текстов вход и текстов изход.

Нов проект: стъпки 5–7

5. Project title: task01. Избери папката Week01.

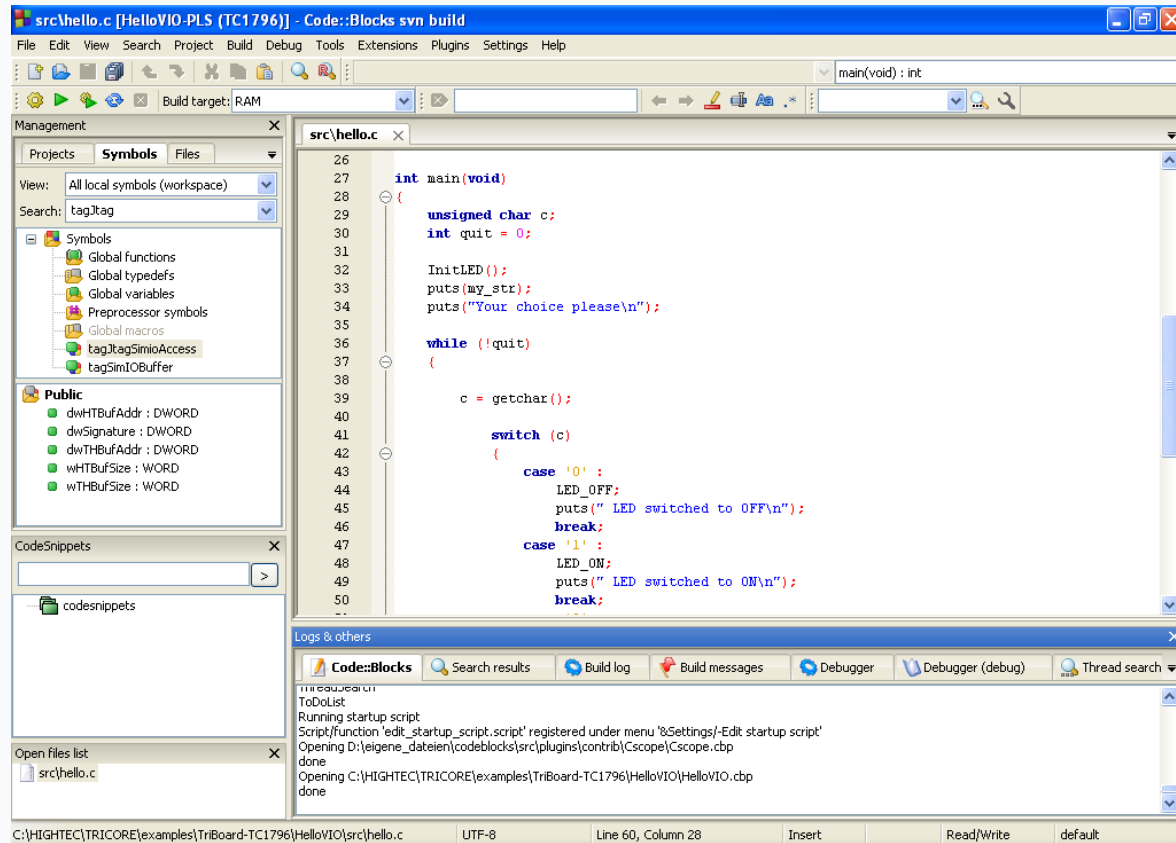
6. Остави GNU GCC Compiler и завърши с Finish.

7. Вляво разгъни Sources и отвори main.cpp с двойно щракване.

Замени примерния код с програмата от следващите слайдове.

За нова задача създавай нов проект, за да пазиш предишното решение.

Къде работим в Code::Blocks



Вляво: проектът и файловете.

В средата: редакторът за кода.

Долу: съобщенията при компилиране.

Конзолата се отваря отделно при стартиране.

Илюстрация от официалното ръководство. Външният вид може да се различава.

Първа работеща програма

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, Olympiad!" << '\n';
    return 0;
}
```

1. Въведи кода в main.cpp.
2. Запази с Ctrl+S.
3. Избери Build > Build and run (обичайно F9).

Очакван изход:
Hello, Olympiad!

Какво правят Build и Run

Build компилира кода и създава изпълним файл.

Run стартира вече създадената програма.

Build and run изпълнява последователно двете действия.

След промяна използвай Build and run. Ако компилацията се провали, първо поправи грешката.

Текст като „Process returned 0“ е съобщение на средата, а не част от твоя cout.

Началният шаблон

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello!" << '\n';
    return 0;
}
```

```
#include <iostream>
Средства за вход и изход.
```

```
using namespace std;
Пишем cin и cout без std::.
```

```
int main() и { }
Начало на изпълнението и
тяло на програмата.
```

```
return 0;
Успешен край на програмата.
```

cout: показване на резултат

```
cout << 2 + 3 << '\n';
```

```
cout << "2 + 3" << '\n';
```

Първият ред пресмята израза.
Изход: 5

Вторият ред извежда текст.
Изход: 2 + 3

Двойните кавички означават текст. Изразът извън тях се пресмята.

Преди стартиране предскажи: какво ще покаже `cout << "5";`?

Нов ред и интервал

```
cout << 16 << 15;  
// 1615
```

```
cout << 16 << ' ' << 15;  
// 16 15
```

```
cout << 16 << '\n' << 15;  
// на два отделни реда
```

' ' е един интервал.

'\n' е нов ред.

// започва коментар. Той е бележка за човека и не се изпълнява.

Разделителят между числата е част от правилния отговор.

Малко упражнение: моята визитка

Напиши програма, която извежда на три отделни реда:

Hello!

I am in grade 5.

I learn C++.

Промени първия ред и стартирай отново.

За първите опити използваме текст на латиница, за да избегнем затруднения с кодировката.

Време за опит: 3 минути. Цел: кавички, ; и нов ред.

Променливи за цели числа

```
int a, b;
```

```
a = 7;
```

```
b = 5;
```

```
cout << a + b << '\n';
```

Променливата е именувано място за стойност.

int означава целочислен тип.

a и b са имена. След присвояването пазят 7 и 5.

Стойността може да се промени при изпълнение.

Не използвай стойността на променлива, преди да я въведеш или зададеш.

cin: въвеждане на данни

```
int a, b;  
cin >> a >> b;  
cout << a + b << '\n';
```

При стартиране програмата чака числа.

Щракни в конзолата, напиши:
7 5
и натисни Enter.

cin записва 7 в a и 5 в b.
Резултатът е 12.

Числата може да са разделени с интервал или нов ред.

Четене на условие

Вход: кои данни получаваме и в какъв ред?

Ограничения: кои стойности са позволени?

Идея: как намираме отговора на хартия?

Изход: колко числа показваме и как ги разделяме?

Пример: един тест, с който проверяваме разбирането си.

Първо описваме идеята с думи. После пишем код.

01 · Две кутии

В първата кутия има а молива, а във втората b. Намери общия брой.

Вход

Две цели числа а и b. $0 \leq a, b \leq 1000$.

Примерен вход

7 5

Изход

Едно цяло число: общият брой.

Примерен изход

12

01 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << a + b << '\n';
    return 0;
}
```

Събираме броя в двете кутии.

Допълнителни тестове:

0 0 → 0

0 5 → 5

1000 1000 → 2000

Проследяване с нов вход

Стартирай задача 01 отново с вход 0 5.

След cin: $a = 0$ и $b = 5$.

Изчислението е $0 + 5$, затова изходът е 5.

Пробвай и 1000 1000. Предскажи резултата преди стартиране.

Кодът трябва да използва $a + b$, а не предварително записаното число 12.

Проверка в края на първия час

1. Какво прави `cin` и къде пишеш входните числа?
2. Каква е разликата между $2 + 3$ и `"2 + 3"`?
3. Защо не извеждаме „Въведи а“ в състезателно решение?
4. Къде е запазен твоят `main.cpp`?

Покажи работещата задача 01 с два различни входа.

Отговори: `cin` чете от конзолата; израз срещу текст; изходът трябва да спазва условието.

Аритметични изрази

В C++ събиране се записва с +, изваждане с – в математиката и с - в кода, умножение с *.

Първо се пресмятат скобите, после умножението, после събирането и изваждането.

$$3 + 4 * 2 = 3 + 8 = 11$$

$$(3 + 4) * 2 = 7 * 2 = 14$$

В кода пишем 2 * a. Записът 2a не е умножение в C++.

Присвояване: нова стойност

```
int x = 5;  
x = x + 2;  
cout << x << '\n';
```

След първия ред: x е 5.

На втория ред първо смятаме $5 + 2$. После записваме 7 в x .

Изходът е 7.

Знакът `=` означава присвояване. Тук той не е математическо уравнение.

Проследяване ред по ред

```
int x = 4;  
int y = x + 3;  
x = 10;  
cout << x << ' ' << y;
```

Преди да стартираш:
Какви са x и y накрая?

1. x получава 4.
2. y получава $4 + 3 = 7$.
3. x получава 10.

Изход: 10 7
y пази получената стойност 7, не формула, която се обновява.

02 · Правоъгълна площадка

Площадка има страни a и b метра. Намери обиколката и лицето.

Вход

Две цели числа a и b . $1 \leq a, b \leq 1000$.

Примерен вход

3 5

Изход

Обиколка и лице на един ред с интервал.

Примерен изход

16 15

Обиколката има нужда от скоби

$$2 * (a + b)$$

// при $a = 3$, $b = 5$:

$$2 * (3 + 5) = 16$$

// грешната формула:

$$2 * 3 + 5 = 11$$

Правоъгълникът има две страни по a и две страни по b .

$$P = a + b + a + b$$

$$P = 2 * (a + b)$$

$$S = a * b$$

Редовете със знак $=$ вляво са математическо проследяване, не готов C++ код.

02 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << 2 * (a + b) << ' '
         << a * b << '\n';
    return 0;
}
```

Обиколка: $a + b + a + b$.
Лице: $a * b$.

Допълнителни тестове:

1	1	→	4	1
2	3	→	10	6
1000	1000	→	4000	1000000

03 · Пакети моливи

Има k пакета с по p молива и още r отделни молива. Колко са общо?

Вход

Цели k, p, r . $0 \leq k, r \leq 1000, 1 \leq p \leq 1000$.

Изход

Едно цяло число: общият брой.

Примерен вход

4 6 3

Примерен изход

27

04 · Време за пътуване

Пътуване продължава h часа и m минути. Изрази продължителността в минути.

Вход

Цели h и m . $0 \leq h \leq 1000$, $0 \leq m \leq 59$.

Примерен вход

2 15

Изход

Едно цяло число: всички минути.

Примерен изход

135

Проверка с различни тестове

Първо пресметни очаквания отговор на хартия.

После стартирай програмата и сравни двата резултата.

За задача 03: 0 6 3 проверява случая без пакети.

За задача 04: 0 59 проверява случая без цели часове.

Използвай и най-големи допустими данни. Не проверявай невалиден вход като страна 0 в задача 02.

Тестът е двойка: входни данни и очакван изход.

Грешка при компилиране

```
int a, b;  
cin >> a >> b  
cout << a + b << '\n';
```

В този код липсва ; след cin.

Прочети първата грешка в Build messages.

Провери посочения ред и реда преди него.

Поправи, запази и компилирай отново. Една пропусната ; може да предизвика няколко съобщения.

Редовете с #include и отварящите/затварящите скоби не получават произволно ;.

Програмата тръгва, но греши

```
int a, b;  
cin >> a >> b;  
cout << "a + b";
```

Кодът се компилира.

Но извежда текста $a + b$.

Поправка:

```
cout << a + b << '\n';
```

Друг пример за логическа грешка е $2 * a + b$ вместо $2 * (a + b)$.

Правилен синтаксис не гарантира правилна идея.

Когато нещо не работи

Прозорецът чака: въведи всички числа, които cin очаква, и натисни Enter.

Виждаш стар резултат: затвори предишното изпълнение, запази и избери Build and run.

„multiple definition of main“: провери дали в проекта няма две програми с main.

Числата са слепени: добави интервал или нов ред според условието.

Грешни посоки: cin >> променлива, а cout << стойност.

Файлове и собствена работа

main.cpp съдържа твоя C++ код. Него пазим и подаваме според указанията на задачата.

Файлът .cbr пази настройките на проекта в Code::Blocks.

Файлът .exe е изпълнимата програма за Windows.

Пази всяка задача в отделен проект. Отвори отново .cbr, за да продължиш работа.

Преди предаване провери дали .cpp е последната работеща версия.

Кратка самопроверка

1. Колко е $3 + 4 * 2$? А $(3 + 4) * 2$?
2. Какво извежда `cout << "3 + 4";`?
3. Как прочитаме две цели числа `a` и `b`?
4. Как извеждаме `a` и `b` с интервал?
5. Какъв файл пазим с изходния код?

Отговори: 11 и 14; текст 3 + 4; `cin >> a >> b; cout << a << ' ' << b; .cpp.`

Самоподготовка след урока

Сесия 1, около 20 минути: довърши 03–04 и провери с по три теста.

Сесия 2, около 20 минути: реши 05 и 06. Използвай шаблона при нужда.

Сесия 3, около 20 минути: поправи грешките и напиши едно решение отново без преписване.

Ако си готов: опитай задачата „Крти в клуба“ и намери собствен граничен тест.

05 · Точки в турнир

В нашия учебен турнир решена задача носи 3 точки, а частично решена 1 точка. Ученик има a решени и b частично решени задачи.

Вход

Цели a и b . $0 \leq a, b \leq 1000$.

Изход

Едно цяло число: сборът от точките.

Примерен вход

4 2

Примерен изход

14

06 · Ресто

Купуваме n тетрадки по p стотинки. Плащаме s стотинки. Намери рестото. Гарантирано е, че парите стигат.

Вход

Цели n, p, s . $1 \leq n, p \leq 1000, n * p \leq s \leq 1000000$.

Примерен вход

3 120 500

Изход

Едно цяло число: рестото в стотинки.

Примерен изход

140

Допълнително: карти в клуба

В клуба има a ученици. Всеки носи b карти, а учителят добавя c карти. Намери общия брой.

Вход

Цели a, b, c . $0 \leq a, b, c \leq 100$.

Примерен вход

5 4 3

Изход

Едно цяло число: броят карти.

Примерен изход

23

Разширение при $b \geq 1$: всеки ученик подарява една карта извън клуба. Колко остават?

Подсказки без готов код

03: колко молива има само в пакетите? После добави отделните.

04: превърни часовете в минути. Прибави оставащите минути.

05: пресметни отделно точките за решени и частично решени задачи.

06: първо намери цената на всички тетрадки.

Карти: всички ученици носят еднакъв брой карти. Кое действие заменя повтарящото се събиране?

Дневник на грешките

Каква грешка направих?

С кой тест я открих?

Какво поправих?

Как ще разпозная същата грешка следващия път?

Пример: „За обиколката получих 11 при вход 3 5. Бях написал $2 * a + b$. Добавих скоби и получих 16.“

Запиши поне една поправка от днешната си работа.

Решения за самопроверка

Следващите слайдове показват готов код.

Преди да ги отвориш, запиши своята идея и направи опит.

След сравняване затвори решението и напиши програмата отново.
Обясни всяко изчисление.

03 - Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int k, p, r;
    cin >> k >> p >> r;
    cout << k * p + r << '\n';
    return 0;
}
```

В пакетите има $k * p$ молива.
Добавяме отделните r .

Допълнителни тестове:

0 6 3 → 3

1 1 0 → 1

1000 1000 1000 → 1001000

04 - Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int h, m;
    cin >> h >> m;
    cout << h * 60 + m << '\n';
    return 0;
}
```

Един час има 60 минути.
2 часа са 120 минути.
 $120 + 15 = 135$.

Допълнителни тестове:

0 0 → 0
0 59 → 59
1 0 → 60

05 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << 3 * a + b << '\n';
    return 0;
}
```

Точките за а задачи са $3 * a$.
За останалите са b.
Правилото е само за тази задача.

Допълнителни тестове:

0 8 → 8

2 0 → 6

1000 1000 → 4000

06 - Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int n, p, s;
    cin >> n >> p >> s;
    cout << s - n * p << '\n';
    return 0;
}
```

Обща цена: $n * p$.
Ресто: платено – цена.
Всички суми са в стотинки.

Допълнителни тестове:

2	50	100	→	0
1	1	10	→	9
1000	1000	1000000	→	0

Карти в клуба: решение

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    cout << a * b + c << '\n';
    return 0;
}
```

Учениците носят общо $a * b$ карти.
Добавяме с от учителя.

Тестове:

5 4 3 → 23

0 4 3 → 3

100 100 100 → 10100

При подаряване на по една карта извън клуба: $a * (b - 1) + c$, когато $b \geq 1$.

Продължение: деление и цифри

Материал за следващото занятие и за самостоятелно надграждане.

Първо завърши уверено задачи 01–06.

Следват /, %, преобразуване на време и задачи 07–12 от работния лист.

Целочислено деление и остатък

$17 / 5 \quad // \quad 3$
 $17 \% 5 \quad // \quad 2$

$15 / 5 \quad // \quad 3$
 $15 \% 5 \quad // \quad 0$

$4 / 5 \quad // \quad 0$
 $4 \% 5 \quad // \quad 4$

17 топчета, по 5 в кутия:
3 пълни кутии и 2 топчета
остават.

Проверка: $17 = 3 * 5 + 2$.

За неотрицателно n и
положително k :
 n / k е броят пълни групи.
 $n \% k$ е остатъкът от 0 до $k - 1$.

Деление на нула е
недопустимо.

07 · Пълни кутии

Имаме n топчета. В една кутия се събират k . Напълни възможно най-много кутии.

Вход

Цели n , k . $0 \leq n \leq 1000000$, $1 \leq k \leq 1000$.

Примерен вход

17 5

Изход

Пълни кутии и останали топчета с интервал.

Примерен изход

3 2

07 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    cout << n / k << ' '
         << n % k << '\n';
    return 0;
}
```

Частното брой пълните кутии.
Остатъкът брой топчетата
извън тях.

Допълнителни тестове:

15 5 → 3 0

4 5 → 0 4

0 5 → 0 0

08 · Часове и минути

Представи продължителност от t минути като цели часове и оставащи минути. Часовете могат да са над 23.

Вход

Цяло t . $0 \leq t \leq 1000000$.

Примерен вход

135

Изход

Часове и минути на един ред с интервал.

Примерен изход

2 15

08 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int t;
    cin >> t;
    cout << t / 60 << ' '
         << t % 60 << '\n';
    return 0;
}
```

На всеки 60 минути имаме
цял час.
Това е продължителност, а не
час от денонощието.

Допълнителни тестове:

59 → 0 59

60 → 1 0

1500 → 25 0

Отделяне на цифри

```
int n = 407;
```

```
n / 100      // 4  
n / 10       // 40  
n / 10 % 10  // 0  
n % 10       // 7
```

Стотиците са 4.
Десетиците са 0.
Единиците са 7.

$n / 10$ премахва единиците.
След това $\% 10$ взема
последната останала цифра.

Затова десетиците са $n / 10$
 $\% 10$, а не само $n / 10$.

09 · Сбор на цифри

Намери сбора на цифрите на трицифрено число n .

Вход

Цяло n . $100 \leq n \leq 999$.

Изход

Едно цяло число: сборът.

Примерен вход

407

Примерен изход

11

09 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int a = n / 100;
    int b = n / 10 % 10;
    int c = n % 10;
    cout << a + b + c << '\n';
    return 0;
}
```

Отделяме всяка цифра.
После събираме трите
стойности.

Допълнителни тестове:

100 → 1
999 → 27
120 → 3

10 · Автобуси

За n деца трябва автобуси с по k места. Намери най-малкия достатъчен брой. За 0 деца трябва 0 автобуса.

Вход

Цели n, k . $0 \leq n \leq 1000000$, $1 \leq k \leq 1000$.

Примерен вход

26 12

Изход

Едно цяло число: броят автобуси.

Примерен изход

3

Защо добавяме $k - 1$?

```
// n = 26, k = 12  
(26 + 12 - 1) / 12  
37 / 12 // 3
```

```
// n = 24, k = 12  
(24 + 12 - 1) / 12  
35 / 12 // 2
```

При непълна група е нужна още една.

Ако $n = q * k$, добавянето на $k - 1$ не създава нова пълна група.

Ако остатъкът r е от 1 до $k - 1$, тогава $r + k - 1$ е поне k и под $2 * k$: получаваме точно още една група.

За $n = 0$ формулата също дава 0.

10 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    cout << (n + k - 1) / k << '\n';
    return 0;
}
```

И непълният автобус е нужен.
При 26 деца два автобуса по
12 места не стигат: $24 < 26$.

Допълнителни тестове:

0 12 → 0
24 12 → 2
25 12 → 3

11 · След една минута

Часовник показва h часа и m минути. Какво ще показва след една минута в 24-часов формат?

Вход

Цели h, m . $0 \leq h \leq 23, 0 \leq m \leq 59$.

Примерен вход

23 59

Изход

Час и минута с интервал, без водещи нули.

Примерен изход

0 0

11 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int h, m;
    cin >> h >> m;
    int t = (h * 60 + m + 1) % 1440;
    cout << t / 60 << ' '
         << t % 60 << '\n';
    return 0;
}
```

Денонощието има 1440 минути.
% 1440 връща 1440 към 0.
После отделяме часа и минутата.

Допълнителни тестове:

12 59	→	13 0
0 0	→	0 1
23 58	→	23 59

12 · Обърнато число

Обърни цифрите на трицифрено n . Изведи резултата като цяло число, без начални нули.

Вход

Цяло n . $100 \leq n \leq 999$.

Примерен вход

305

Изход

Едно цяло число. Например 120 става 21.

Примерен изход

503

12 · Решение и проверка

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int a = n / 100;
    int b = n / 10 % 10;
    int c = n % 10;
    cout << 100 * c + 10 * b + a << '\n';
    return 0;
}
```

Единиците стават стотици.
Десетиците остават в
средата.
Стотиците стават единици.

Допълнителни тестове:

120 → 21

100 → 1

999 → 999

Готов ли съм за следващата тема?

Създавам проект и намирам `main.cpp`.

Обяснявам какво чете `cin` и какво показва `cout`.

Решавам сам поне три от задачи 03–06 и обяснявам формулите.

Подбирам малък, граничен и обикновен тест.

Поправям грешката и запазвам работещото решение.

Ако нещо още е трудно, повтори сходна задача с други числа и запиши точния си въпрос.

Източници и полезни връзки

Материали по плана: „Първи два часа и разширение“, „Работен лист за ученици, седмица 1“ и програмата за 2026/2027.

Изтегляне на Code::Blocks:

<https://www.codeblocks.org/downloads/binaries/>

Официално ръководство и изображение на средата:

https://www.codeblocks.org/docs/main_codeblocks_en3.html

Проверка на инсталационната информация: 23.09.2026. Упражненията са учебни авторски задачи.